SYSTEMS MANAGEMENT



GraphQL APIs for OpenBMC Metrics

OpenRMC-DM as Proxy GraphQL Server

Avinash Natarajan, Senior Principal, Celestica

Navinsennivel Nallasivam, Staff, Celestica

OCP PLATINUM Member

Problem: Redfish Chattiness, and its impact at Rack scale

- Redfish is inherently chatty, and at multi-rack scale, there is value in looking for optimization.
- Telemetry services is a good solution, but among Vendors that support Redfish, depth and behavior of telemetry support vary.
- Citing a couple of problems with models that exist today, which compound at a Multi-Rack scale.
  - Runtime Inefficiency
    - Retrieving many individual sensor resources requires many GETs.
    - Each GET will return considerably more information than the what client needs.
  - Vendor onboarding friction
    - Redfish provides a standard data model, but implementation capabilities vary.
    - Telemetry Service / MetricReport functionality cannot necessarily be assumed across every target platform.
    - Redfish clients need to do basic scalar Redfish access (chatty/inefficient), or request Vendors to implement extensions (Friction)

Challenges: Heterogeneous BMC Telemetry in a Multi-Vendor Rack

⚠ Problem 1: Runtime Inefficiency
N×M individual GET requests per poll cycle
Each response returns full JSON payload — **client needs only Reading (~10 bytes)**

North Bound Application
(Orchestration / Monitoring)

GET /Chassis
GET /Chassis/{id} **× N**
GET /Chassis/{id}/Sensors **× N**
GET /Sensors/{id} **× N×M**

RACK

Scale Up Switch BMC

✗ Telemetry Service
NOT implemented

Sensors (Redfish /Sensors/...)

/Chassis/1/Sensors/port_temp
/Chassis/1/Sensors/asic_temp
/Chassis/1/Sensors/link_util

Compute Node BMC

✓ Telemetry Service
Supported

Sensors (Redfish /Sensors/...)

/Chassis/1/Sensors/CPU_Temp
/Chassis/1/Sensors/inlet_temp
/Chassis/1/Sensors/mem_temp
/Chassis/1/Sensors/fan_speed

Power Shelf (PDU) BMC

~ Telemetry Service
Limited Metric coverage

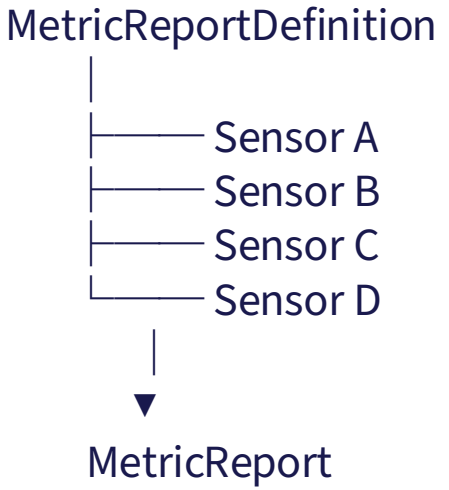
Sensors (Redfish /Sensors/...)

/Chassis/1/Sensors/psu_voltage
/Chassis/1/Sensors/psu_current
/Chassis/1/Sensors/psu_temp

⚠ Problem 2: Vendor Onboarding Friction — Telemetry support varies across BMC vendors · Clients cannot assume MetricReport availability · Must fall back to chatty scalar Redfish access

Telemetry Services: What exists today, and its limitations

- Redfish Telemetry Services exist today for multi-metrics support, with both pull and push models defined.



- Challenge is, when integrating with a heterogeneous set of Vendor BMCs (as is the case with Rack Management), a consistent level of Telemetry service is not guaranteed.

Graph-Query Language: Introduction

- GraphQL is an API query language where the client asks for exactly what it wants, thereby controlling the shape of the response.
- Capability equivalent to Telemetry Services is designed into GraphQL, and is not an optional feature.
- A couple of sample queries/responses comparing Redfish and GraphQL below.

REST / Redfish

```
GET /Sensors/CPU_Temp

Returns:
{
 Id,
 Name,
 Reading,
 ReadingUnits,
 Status,
 Thresholds,
 ...
}
```

</>GraphQL

```
sensor(name: "CPU_Temp") {
  reading
}

Returns:
{
 reading: 36
}
```

</>GraphQL

```
devices(ids: [BMC1, BMC2]) {
  chassis {
    sensors {
      name
      reading
    }
  }
}
```

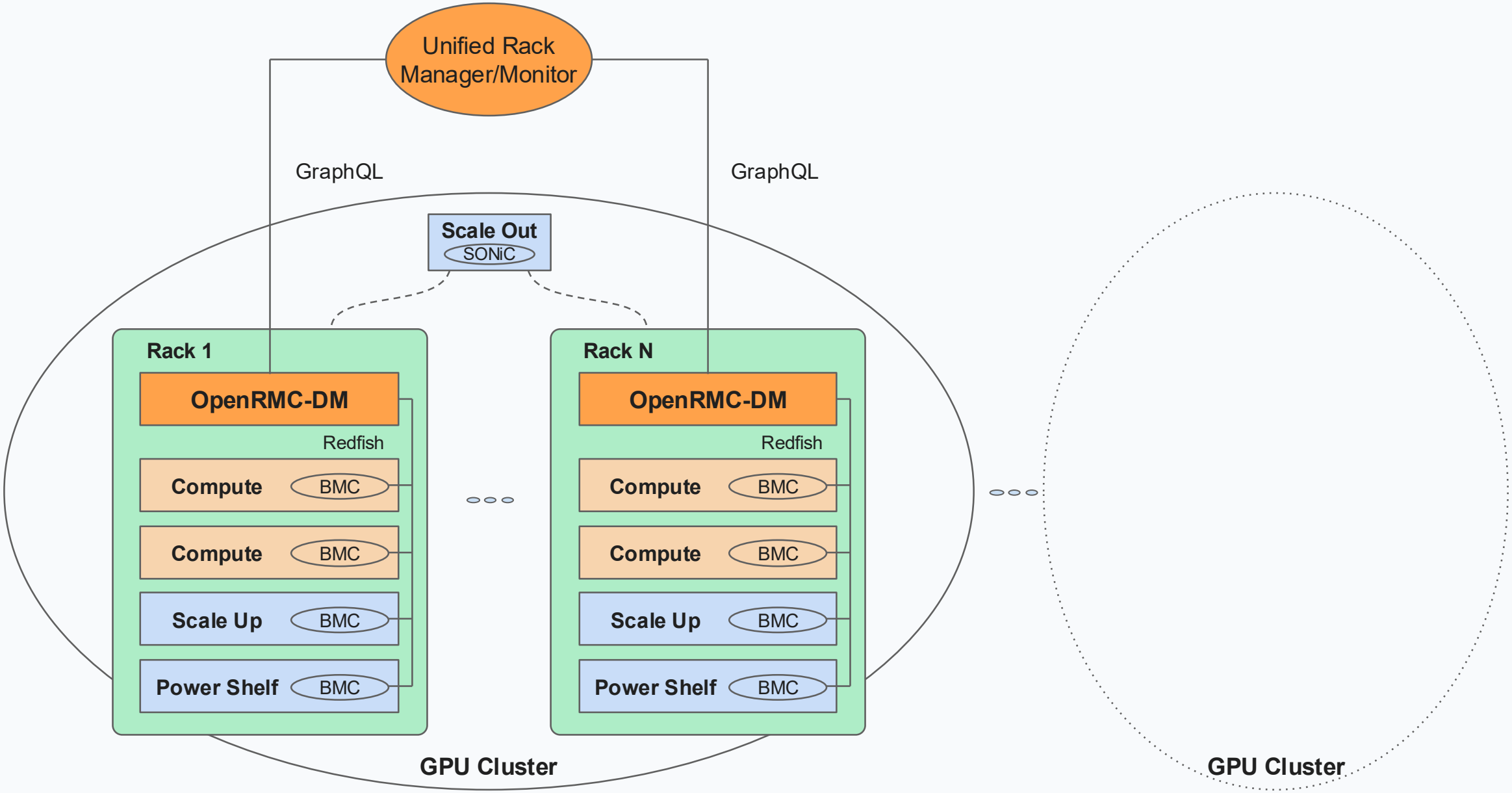
Graph-QL APIs for OpenBMC Metrics

- An OpenBMC metric access modeled around GraphQL would
  - Reduce number of outbound API calls from client, reduce chattiness
  - Prevent over-fetching
  - Reduce client side traversal/aggregation logic
- Factoring in what exists today, a complete shift to Graph-QL for OpenBMC Metrics may not be viable, and may not be the right thing to do either.
- A middle ground might be worth exploring.
- More on that in the upcoming slides…

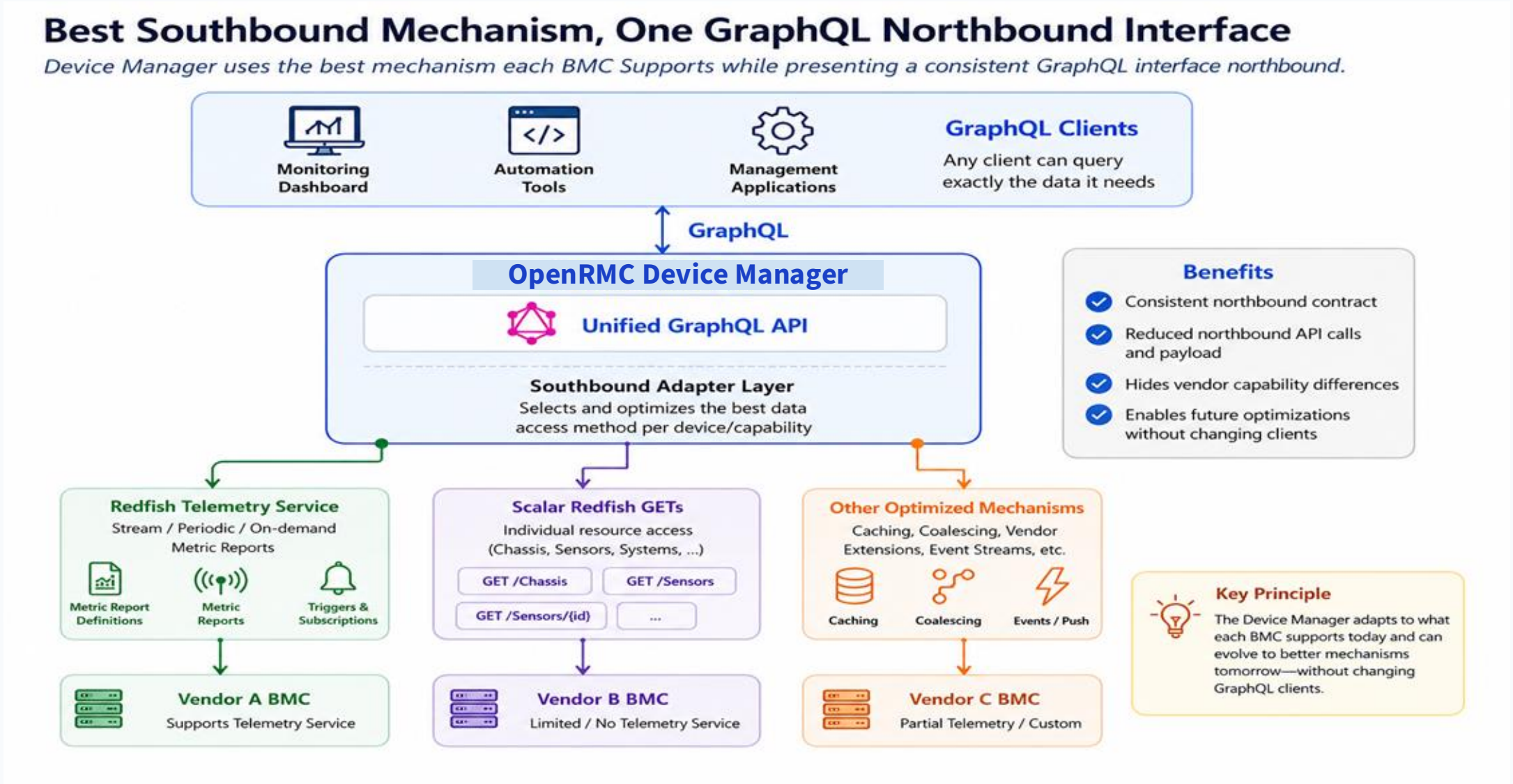
OpenRMC-DM as a Proxy Graph-QL Server

- Open Rack Management Controller – Device Manager (OpenRMC-DM) provides a common abstraction layer to access BMC end points of multiple devices over Redfish API.
- Proposal is for the DM, or an equivalent of a DM to act as a Proxy GraphQL Server
  - This would provide a consistent North bound data access model independent of differences in the underlying BMC Redfish telemetry capabilities.
  - This needs no change to Vendor's BMC Redfish implementation

OpenRMC-DM as a Proxy Graph-QL Server ...Cntd...

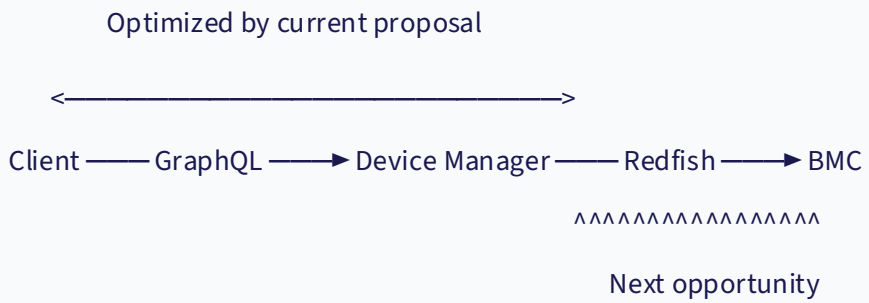


OpenRMC-DM as a Proxy Graph-QL Server ...Cntd...



What GraphQL buys us (and doesn't)

- What it buys us
  - Reduced Vendor onboarding friction
    - The northbound contract is the DM's GraphQL schema rather than each BMC's exact telemetry implementation
  - Reduced Northbound API chattiness
    - One GraphQL query can represent requests that otherwise require multiple Redfish transactions.
  - Reduced Northbound data transfer
    - Clients request only the fields they require.
- What it doesn't buy us
  - While this proposal optimizes Client to DM conversation via a GraphQL proxy, the DM->BMC conversation remains as is with room for optimization



GraphQL vs Redfish — Performance Study

What We Have

- **2 BMC devices** managed by Device Manager
- **30 chassis** across both BMCs (15 each)
- **390 sensors** across all chassis
- Device Manager acts as the Redfish proxy — all NB client requests go through DM

What the Client Wants

All sensor readings across all chassis across both BMC devices

```
Request goal: Just the "Reading" value per sensor, nothing else.
```

Test Methodology

- Both approaches make sequential calls — **no client-side parallelism**
- **Metrics:** API call count and payload bytes (req + resp) at the client boundary
- Wall-clock time is excluded (reflects server processing, not API model)



Redfish Approach — What the Client Must Do

Step 1: `GET /redfish/v1/Chassis` → 1 call returns all 30 chassis IDs (client discards all other fields)

Step 2: `GET /redfish/v1/Chassis/{id}/Sensors` × 30 chassis → 30 calls return sensor ID list (client needs only IDs)

Step 3: `GET /redfish/v1/Chassis/{id}/Sensors/{sid}` × 390 → 390 calls return full sensor JSON (client needs only "Reading")

421
TOTAL API CALLS

222.6KB
DATA TRANSFERRED

GraphQL vs Redfish — GraphQL Performance Results

GraphQL Approach — What the Client Sends

Call 1: `POST /graphql chassisCollection(uuids: ["uuid1", "uuid2"])`
→ Returns all 30 chassis IDs + 390 sensor names in one response

Call 2: `POST /graphql sensorReadings(devices: [30 chassis with 390 sensors])`
→ Returns only the Reading value for all 390 sensors

Total Execution: 2 calls · 66.9 KB transferred

Why the Payload Difference is Large

Redfish Bottleneck: Returns full resource JSON for every call regardless of client needs. A single sensor response is ~450 bytes, but client only uses ~10 bytes (the Reading value).

GraphQL Efficiency: Field selection ensures only `name` and `reading` are returned per sensor, eliminating overhead.

Measured Results (30 chassis · 390 sensors · 2 BMC devices)

Metric	Redfish REST	GraphQL (Iter 1)	Improvement
Client API calls	421	2	**210× fewer**
Response payload	227,951 B	54,625 B	**4.2× less**
Request payload	0 B	13,924 B	—
Total bytes (req + resp)	227,951 B (222.6 KB)	**68,549 B** (66.9 KB)	**3.3× less overall transfer**

An existing alternative - Redpath; Redpath vs GraphQL

CLIENT-SIDE TRAVERSAL

Redpath (DMTF DSP0285)

Borrows XPath syntax to navigate Redfish trees. Resolved **entirely by the client library** — server sees standard raw Redfish calls.

```
/Chassis[*]/Sensors[ReadingType=Temperature]/Reading
```

Internal Traversal Executed by Client Library:

1. `GET /redfish/v1/Chassis`
2. `GET /redfish/v1/Chassis/{id}` × N
3. `GET /redfish/v1/Chassis/{id}/Sensors` × N
4. `GET /redfish/v1/Chassis/{id}/Sensors/{sid}` × N×M
   └ *client-side filter applied after fetch*

* Note:Fewer calls are possible only if the server implements optional Redfish query parameters.

DEVELOPER CONVENIENCE

Redpath = Abstraction Only

Cleaner syntax for the client author, but retains the exact same high wire cost, round trips, and bandwidth overhead as raw Redfish REST.

High Wire Cost • Multiple Round Trips • Heavy Payload

WIRE EFFICIENCY

GraphQL = True Optimization

True wire optimization providing exponentially fewer API calls, drastically smaller payloads, and precise server-side field selection.

Single Query • Server Filtering • Minimal Payload

Call to Action

- Join the OpenRMC-DM sub-project.
  - OpenRMC-DM Subproject: https://www.opencompute.org/projects/openrmc-dm
- Contribute to a Redfish Schema to GraphQL Schema translation tool.
- Expansion of OpenRMC-DM's scope to include a GraphQL northbound variant and the corresponding adaptation layer towards the BMCs, or creation of a new SubProject to take this idea forward if that makes more sense.

Thank You!

